立讯项目-LeRobot 回放服务：数据交互与接口设计

本文档整合后端存储设计、实现说明与接口文档，聚焦**数据交互流程**、**交互设计**与**接口规范**，供前后端联调与集成使用。

参考实际数据目录：`IROS_act_visuotactile_v2_200_next_fps10_A3_test10/IROS_act_visuotactile_v2_200_next_fps10_A3_test10/`

一、交互设计

1.1 三个核心概念

概念	含义	对应接口/数据
数据集	整个 repo：`org/dataset`，包含多集（episode_0, episode_1, …）	**GET info**：整份数据集的元信息
一集（episode）	一条轨迹/一次录制，含多帧（多时间点）	**GET episodes/:episodeId**：该集的整集数据（所有帧 + 视频 URL）
一帧/时间点	某一时刻的状态、动作等	无独立接口：含于单集回放数据中，由前端用 `currentTime` 在集内定位

1.2 一集内播放（单条轨迹按时间走）

- 前端**只调一次**：`GET .../episodes/0`，得到该集的 **EpisodeData**（含 `flatChartData`、`duration`、`videosInfo`）。

- 这份数据已包含该集**所有帧**。播放时用**一个** `currentTime`（0 ～ duration）驱动：视频进度与图表游标均由前端在此份数据上按 `currentTime` 取当前帧展示。

- **结论**：一集内是**前端本地按时间轴播放**，不按帧调接口。

1.3 多集"挨个播放"

- 每切到一集就请求**该集的单集回放数据**一次，然后在该集内用 `currentTime` 按时间播放。

1.4 前后端职责划分

职责	前端	服务端
数据集列表	调用 GET /api/v1/datasets	扫描 DATA_ROOT，返回摘要列表
~~片段列表~~	~~调用 GET episodes，用于展示集数列表~~	~~读 meta/episodes parquet，汇总返回~~
单集回放负载	调用 GET episodes/:id，得到后直接喂给 DataReplayView	读 data parquet（过滤 episode_index），构建 flatChartData（含 task_index）、videosInfo、duration、tasks，返回 EpisodeData

二、存储与数据结构概要（v3.0）

2.1 存储根与单数据集路径

- **DATA_ROOT**：环境变量，默认 `server/data`。

- 单数据集：`{DATA_ROOT}/{org}/{dataset}/`

实际示例（org = dataset = `IROS_act_visuotactile_v2_200_next_fps10_A3_test10`）：

代码块

```
IROS_act_visuotactile_v2_200_next_fps10_A3_test10/
  IROS_act_visuotactile_v2_200_next_fps10_A3_test10/
    meta/
      info.json
      stats.json
      tasks.parquet
      episodes/
        chunk-000/
          file-000.parquet          ← 各集摘要: episode_index, tasks, length 等
    data/
      chunk-000/
        file-000.parquet            ← 多集的帧数据合并，含 episode_index 列
    videos/
      observation.images.cam0/
        chunk-000/
          file-000.mp4              ← 该 chunk 内所有集的视频连续拼接
      observation.images.cam1/chunk-000/file-000.mp4
      observation.images.cam3/chunk-000/file-000.mp4
      observation.images.cam4/chunk-000/file-000.mp4
    debug_logs/                     ← 调试日志（非必需）
```

2.2 路径模板（meta/info.json 中定义）

代码块

```
{
  "data_path":  "data/chunk-{chunk_index:03d}/file-{file_index:03d}.parquet",
  "video_path": "videos/{video_key}/chunk-{chunk_index:03d}/file-{file_index:03d}.mp4"
}
```

索引计算（与 v2.1 不同）：

- `chunk_index` / `file_index` 决定“这一集的数据和视频大概在哪个文件里”。
- 具体到一帧/一集，还是要靠 parquet 里的 `episode_index`、`timestamp` 去过滤和对齐。

变量	解释	计算方式
`chunk_index`	第几个 chunk（块），按 episode 的编号分组 例如 chunks_size = 1000 时： • episode 0–999 → chunk_index = 0 • episode 1000–1999 → chunk_index = 1	`floor(episode_index / chunks_size)`，3 位补零
`file_index`		与 `chunk_index` 相同（每个 chunk 目录下只有一个 file）
`video_key`		features 中 dtype=video 的 key，如 `observation.images.cam0`

v3.0 与 v2.1 路径对比：

版本	数据 parquet	视频
v2.1	`data/{episode_chunk:03d}/episode_{episode_index:06d}.parquet`（每集一文件）	`videos/{video_key}/{episode_chunk:03d}/episode_{episode_index:06d}.mp4`
v3.0	`data/chunk-{chunk_index:03d}/file-{file_index:03d}.parquet`（多集合并，按 episode_index 列过滤）	`videos/{video_key}/chunk-{chunk_index:03d}/file-{file_index:03d}.mp4`

2.3 meta/info.json 真实字段（v3.0）

代码块

```
{
  "codebase_version": "v3.0",
  "robot_type": "realman_robot",
  "total_episodes": 1,
```

```
  "total_frames": 247,
  "total_tasks": 1,
  "chunks_size": 1000,
  "data_files_size_in_mb": 100,
  "video_files_size_in_mb": 500,
  "fps": 10,
  "splits": { "train": "0:1" },
  "data_path": "data/chunk-{chunk_index:03d}/file-{file_index:03d}.parquet",
  "video_path": "videos/{video_key}/chunk-{chunk_index:03d}/file-{file_index:03d}.mp4",
  "features": {
    "action": {
      "dtype": "float32",
      "shape": [8],
      "names": [
        "left_arm_joint_0", "left_arm_joint_1", "left_arm_joint_2",
        "left_arm_joint_3", "left_arm_joint_4", "left_arm_joint_5",
        "left_arm_joint_6", "left_arm_gripper"
      ]
    },
    "observation.state": {
      "dtype": "float32",
      "shape": [7],
      "names": [
        "left_arm_joint_0", "left_arm_joint_1", "left_arm_joint_2",
        "left_arm_joint_3", "left_arm_joint_4", "left_arm_joint_5",
        "left_arm_joint_6"
      ]
    },
    "observation.images.cam0": {
      "dtype": "video",
      "shape": [480, 640, 3],
      "names": ["height", "width", "channels"],
      "info": {
        "video.height": 480, "video.width": 640,
        "video.codec": "av1", "video.pix_fmt": "yuv420p",
        "video.fps": 10, "video.channels": 3, "has_audio": false
      }
    },
    "observation.images.cam1": { "dtype": "video", "shape": [480, 640, 3], "...": "同 cam0" },
    "observation.images.cam3": { "dtype": "video", "shape": [480, 640, 3], "...": "同 cam0" },
    "observation.images.cam4": { "dtype": "video", "shape": [480, 640, 3], "...": "同 cam0" },
    "timestamp":     { "dtype": "float32", "shape": [1], "names": null },
    "frame_index":   { "dtype": "int64",   "shape": [1], "names": null },
```

```
      "episode_index": { "dtype": "int64",   "shape": [1], "names": null },
      "index":         { "dtype": "int64",   "shape": [1], "names": null },
      "task_index":    { "dtype": "int64",   "shape": [1], "names": null }
    }
  }
```

字段说明（结合当前数据集）：

- **codebase_version**：LeRobot 存储格式版本，本数据集为 `v3.0`，决定 data/video 路径和 meta 结构。

- **robot_type**：机器人类型，例如 `realman_robot`，前端可用于界面展示或做设备区分。

- **total_episodes / total_frames / total_tasks**：数据集中 episode 总数、总帧数、任务总数，供列表统计与校验使用。

- **chunks_size**：每个 chunk 能容纳的 episode 数量，服务端用 `chunk_index = floor(episodeId / chunks_size)` 计算出应访问的 `chunk-XXX/file-XXX`。

- **data_files_size_in_mb / video_files_size_in_mb**：数据/视频文件的总体积（MB），用作容量展示或监控。

- **fps**：采样帧率（每秒帧数），与 `timestamp`、视频帧率一起决定时间轴。

- **splits**：数据集划分信息（如 `train`, `val`，当前为 `\"train\": \"0:1\"`），用于训练/评估划分。

- **data_path**：数值数据 parquet 的路径模板，包含 `{chunk_index}` 和 `{file_index}` 占位符，服务端按此拼出 `data/chunk-000/file-000.parquet` 等实际路径。

- **video_path**：视频文件的路径模板，包含 `{video_key}`（如 `observation.images.cam0`）和 `{chunk_index}`、`{file_index}`，服务端用来定位 `videos/.../chunk-000/file-000.mp4`。

- **features**：各特征列（action、observation.state、各路相机、timestamp、index 等）的元信息，是解析一帧数据的“字典”：

  - **action**：8 维浮点向量，`names` 对应 7 个关节 + 1 个 gripper，服务端用来展开成 `action_left_arm_joint_0` 等键。

  - **observation.state**：7 维浮点向量，表示关节状态，`names` 与 action 前 7 维对齐。

  - **observation.images.cam***：4 路视频特征，给出分辨率、通道数、编码、fps 等，服务端据此构造 `videosInfo` 和前端 `<video>` 布局信息。

  - **timestamp**：每帧时间戳（秒），shape 为 `[1]`，驱动回放时间轴与视频同步。

  - **frame_index / episode_index / index**：帧内索引、所属 episode 索引、全局索引，主要用于排序和过滤。

  - **task_index**：每帧所属任务阶段编号（整数），结合 `meta/tasks.parquet` / `tasks[]` 字段驱动“执行状态展示”面板。

2.4 时间与视频对齐

- 数据侧：`timestamp` = episode 内秒数（0 ～ duration），与视频进度对齐。
- v3.0 chunk 视频包含多集；服务端需从 `meta/episodes` parquet 中读取每集的起止帧，换算为视频时间偏移 `segmentStart` / `segmentEnd`，前端 seek 时加上 offset。
- 前端用一条 `currentTime`（0 ～ duration）驱动：`video.currentTime = segmentStart + currentTime`。

三、接口文档

基础 URL：由部署决定，示例 `http://localhost:4000`。

API 前缀：`/api/v1`。

3.1 🚀列出所有数据集

扫描 DATA_ROOT 下所有有效数据集（含 `meta/info.json`），返回摘要列表。

方法	路径	说明
GET	`/api/v1/datasets`	列出所有数据集

查询参数（可选）

参数	类型	默认	说明
page	number	1	分页页码

page_size	number	20	每页条数

请求示例

代码块

```
GET /api/v1/datasets HTTP/1.1
Host: localhost:4000
```

成功：200 OK

代码块


```
{
  "total": 1,
  "page": 1,
  "page_size": 20,
  "datasets": [
    {
      "repoId": "IROS_act_visuotactile_v2_200_next_fps10_A3_test10/IROS_act_visuotactile_v2_200_next_fps10_A3_test10",
      "robot_type": "realman_robot",
      "codebase_version": "v3.0",
      "total_episodes": 1,
      "total_frames": 247,
      "fps": 10,
      "total_tasks": 1,
      "data_files_size_in_mb": 100,
      "video_files_size_in_mb": 500,
      "episodes": [
        { "episode_index": 0, "length": 247, "duration_sec": 24.7, "task": "Place the cylinder into the A3 hole." }
      ]
    }
  ]
}
```

返回值说明

字段	类型	说明
total	number	数据集总数
datasets	array	数据集摘要列表
datasets[].repoId	string	数据集标识，格式 `org/dataset`
datasets[].robot_type	string	机器人类型，如 `realman_robot`
datasets[].codebase_version	string	LeRobot 版本，如 `v3.0`
datasets[].total_episodes	number	episode 总数
datasets[].total_frames	number	总帧数
datasets[].fps	number	帧率
datasets[].data_files_size_in_mb	number	数据文件大小（MB）
datasets[].video_files_size_in_mb	number	视频文件大小（MB）
datasets[].episodes	array	各集摘要，含 episode_index、length帧数、duration_sec、task

失败

状态码	说明
500	服务端错误（如 DATA_ROOT 不存在）

3.2 获取片段列表

读取 `meta/episodes/` 下的 parquet 文件，返回每个 episode 的摘要（帧数、时长、任务描述等）。

方法	路径	说明
GET	`/api/v1/datasets/:org/:dataset/episodes`	获取数据集的片段列表

路径参数

参数	类型	必填	说明
org	string	是	组织/目录名
dataset	string	是	数据集名

请求示例

代码块

```
GET /api/v1/datasets/IROS_act_visuotactile_v2_200_next_fps10_A3_test10/IROS_act_visuotactile_v2_200_next_fps10_A3_test10/episodes HTTP/1.1
Host: localhost:4000
```

成功：200 OK

代码块

```
{
  "total": 1,
  "episodes": [
    {
      "episode_index": 0,
      "task_index": 0,
      "task": "Place the cylinder into the A3 hole.",
      "length": 247,
      "duration_sec": 24.7
    }
  ]
}
```

返回值说明

字段	类型	说明
total	number	episode 总数
episodes	array	片段摘要列表
episodes[].episode_index	number	episode 序号，从 0 开始
episodes[].task_index	number	对应任务编号
episodes[].task	string	任务描述文本
episodes[].length	number	该 episode 的帧数
episodes[].duration_sec	number	该 episode 的时长（秒），= length / fps

失败

状态码	说明

404	数据集不存在
500	服务端错误

3.3 🚀 获取单集回放数据（EpisodeData）

返回该集回放所需完整负载，前端可直接用于 DataReplayView。

服务端流程：

1. 读 `meta/info.json`，计算 `chunk_index = floor(episodeId / chunks_size)`，`file_index = chunk_index`。
2. 读 `data/chunk-{chunk_index:03d}/file-{file_index:03d}.parquet`，过滤 `episode_index == episodeId`。
3. 构建 `flatChartData`（含 `obs_state_*`、`action_*`、`timestamp` 键）。
4. 读 `meta/episodes/chunk-{chunk_index:03d}/file-{file_index:03d}.parquet`，获取该集在 chunk 视频中的起止帧，换算为 `segmentStart` / `segmentEnd`。
5. 按 features 中 dtype=video 的键构建 `videosInfo`，URL 指向 4.6 节接口。
6. 读 `meta/tasks.parquet`，按 `task_index` 升序排列，构建 `tasks` 字符串数组（`tasks[i]` = task_index 为 i 的任务描述文本）。

方法	路径	说明
GET	`/api/v1/datasets/:org/:dataset/episodes/:episodeId`	获取指定 episode 的回放数据

路径参数

参数	类型	必填	说明

org	string	是	组织/目录名
dataset	string	是	数据集名
episodeId	number	是	Episode 序号，从 0 开始

请求示例

代码块

```
GET /api/v1/datasets/IROS_act_visuotactile_v2_200_next_fps10_A3_test10/IROS_act_visuotactile_v2_200_next_fps10_A3_test10/episodes/0 HTTP/1.1
Host: localhost:4000
```

成功：200 OK

代码块

```
{
  "datasetInfo": {
    "repoId": "IROS_act_visuotactile_v2_200_next_fps10_A3_test10/IROS_act_visuotactile_v2_200_next_fps10_A3_test10",
    "robot_type": "realman_robot",
    "codebase_version": "v3.0",
    "total_episodes": 1,
    "total_frames": 247,
    "fps": 10,
    "total_tasks": 1,
    "data_files_size_in_mb": 100,
    "video_files_size_in_mb": 500,
    "cameras": [
      { "name": "observation.images.cam0", "width": 640, "height": 480, "codec": "av1" },
      { "name": "observation.images.cam1", "width": 640, "height": 480, "codec": "av1" },
      { "name": "observation.images.cam3", "width": 640, "height": 480, "codec": "av1" },
```

```
        { "name": "observation.images.cam4", "width": 640, "height": 480, "codec": "av1" }
      ]
    },
    "episodeId": 0,
    "videosInfo": [
      {
        "filename": "observation.images.cam0",
        "url": "http://localhost:4000/api/v1/datasets/IROS_.../episodes/0/videos/observation.images.cam0",
        "isSegmented": false,
        "segmentStart": 0.0,
        "segmentEnd": 24.7,
        "segmentDuration": 24.7
      },
      {
        "filename": "observation.images.cam1",
        "url": "http://localhost:4000/api/v1/.../videos/observation.images.cam1",
        "isSegmented": false,
        "segmentStart": 0.0,
        "segmentEnd": 24.7,
        "segmentDuration": 24.7
      }
    ],
    "flatChartData": [
      {
        "timestamp": 0.0,
        "task_index": 0,
        "obs_state_left_arm_joint_0": 23.4,
        "obs_state_left_arm_joint_1": 61.2,
        "obs_state_left_arm_joint_2": 35.1,
        "obs_state_left_arm_joint_3": 67.8,
        "obs_state_left_arm_joint_4": 17.3,
        "obs_state_left_arm_joint_5": 84.2,
        "obs_state_left_arm_joint_6": 75.1,
        "obs_state_right_arm_joint_0": 23.4,
        "obs_state_right_arm_joint_1": 61.2,
        "obs_state_right_arm_joint_2": 35.1,
        "obs_state_right_arm_joint_3": 67.8,
        "obs_state_right_arm_joint_4": 17.3,
        "obs_state_right_arm_joint_5": 84.2,
        "obs_state_right_arm_joint_6": 75.1,
        "action_left_arm_joint_0": 23.6,
        "action_left_arm_joint_1": 61.4,
        "action_left_arm_joint_2": 35.3,
        "action_left_arm_joint_3": 68.0,
```

```
        "action_left_arm_joint_4": 17.4,
        "action_left_arm_joint_5": 84.4,
        "action_left_arm_joint_6": 75.2,
        "action_right_arm_joint_0": 23.6,
        "action_right_arm_joint_1": 61.4,
        "action_right_arm_joint_2": 35.3,
        "action_right_arm_joint_3": 68.0,
        "action_right_arm_joint_4": 17.4,
        "action_right_arm_joint_5": 84.4,
        "action_right_arm_joint_6": 75.2,
        "action_right_arm_gripper": 0.0
      }
    ],
    "episodes": [0],
    "ignoredColumns": ["index", "episode_index", "frame_index"],
    "duration": 24.7,
    "task": "Place the cylinder into the A3 hole.",
  }
```

flatChartData 命名约定

parquet 列名	flatChartData 键名	说明
`observation.state[i]`	`obs_state_{names[i]}`	关节状态，下标对应 features.names
`action[i]`	`action_{names[i]}`	动作输出，下标对应 features.names
`timestamp`	`timestamp`	帧时间戳（秒）
`task_index`	`task_index`	当前帧任务阶段编号，驱动"执行状态展示"面板

例如 `observation.state` features.names = `["left_arm_joint_0", ..., "left_arm_joint_6"]`，则第 0 维展开为键 `obs_state_left_arm_joint_0`。

返回值说明

字段	类型	说明
datasetInfo	object	数据集展示用信息
episodeId	number	当前 episode 序号
videosInfo	array	视频列表，每项含 url（指向 4.6 节）、segmentStart/End（v3 chunk 内偏移）
flatChartData	array	扁平图表数据，每项含 `timestamp`、`task_index` 及各 series 键值对
episodes	number[]	当前数据集可选的 episode 序号列表
ignoredColumns	string[]	不参与图表渲染的 parquet 列名
duration	number	本 episode 时长（秒）
task	string	当前 episode 整体任务描述文本（来自 meta/tasks.parquet）

| tasks | string[] | **任务名称列表**，下标即 `task_index`，用于"执行状态展示"面板。来自 meta/tasks.parquet，按 task_index 升序排列 |

flatChartData 帧结构中的 task_index

字段	类型	说明
task_index	number	该帧所处的任务阶段编号，与 `tasks[]` 数组下标对应

> **数据来源与渲染逻辑**

> - `meta/tasks.parquet`：存储任务索引与描述文本映射（如 `{task_index: 0, task: "Place the cylinder..."}`）
> - `data/*.parquet`：每帧含 `task_index`，指示当前帧属于哪个任务阶段
> - 后端将 tasks.parquet 按 task_index 排序后，以字符串数组形式放入 `tasks` 字段返回

> 前端渲染逻辑

> - 读取当前帧（`flatChartData[currentFrameIndex]`）的 `task_index`
> - `tasks[task_index]` 对应的条目为活跃步骤（转动图标 + 高亮）
> - 下标 < task_index → 已完成（✓ 绿色边框）；下标 > task_index → 待执行（半透明）
> - 点击步骤：seek 到 flatChartData 中该 task_index 首次出现帧的 `timestamp`

videosInfo[] 每项

字段	类型	说明
filename	string	相机键名（与 info.features 中 dtype=video 的 key 一致）
url	string	该路视频流完整 URL（指向 4.6 节接口）
isSegmented	boolean	v3.0 单集对应 chunk 文件中的片段时为 true，否则 false
segmentStart	number	片段在 chunk 视频中的起始时间（秒），前端 seek 时加此偏移
segmentEnd	number	片段结束时间（秒）
segmentDuration	number	片段时长（秒），即 duration

失败

状态码	说明
404	数据集或该 episode 不存在
500	服务端错误（如 parquet 损坏）

3.4 🚀 获取单路视频流

用于回放某 episode 的某一路相机视频；`videosInfo[].url` 即指向此接口。

服务端将根据 `info.json` 的 `video_path` 模板和 `chunks_size` 计算出实际文件路径：

代码块

```
videos/{video_key}/chunk-{chunk_index:03d}/file-{file_index:03d}.mp4
```

并以流的形式返回（支持 `Range`）。

方法	路径	说明
GET	`/api/v1/datasets/:org/:dataset/episodes/:episodeId/videos/:camera`	获取指定 episode、指定相机的视频流

路径参数

参数	类型	必填	说明
org	string	是	组织/目录名
dataset	string	是	数据集名

episodeId	number	是	Episode 序号
camera	string	是	相机键名（features 中 dtype=video 的 key），如 `observation.images.cam0`

请求示例

代码块

```
GET /api/v1/datasets/IROS_act_visuotactile_v2_200_next_fps10_A3_test10/IROS_act_visuotactile_v2_200_next_fps10_A3_test10/episodes/0/videos/observation.images.cam0 HTTP/1.1
Host: localhost:4000
Range: bytes=0-
```

成功：200 OK 或 206 Partial Content（带 `Range` 时）

类型	说明
Content-Type	`video/mp4`
Accept-Ranges	`bytes`
Body	视频二进制流（完整或 Range 指定区间）

> **注意：** v3.0 chunk 文件包含多集视频，前端应使用 `videosInfo[].segmentStart` 作为初始 seek 位置（`video.currentTime = segmentStart + localTime`）。

失败

状态码	说明
404	数据集、episode 或该相机视频不存在
500	服务端错误

实时监控页面接口文档

> 依赖服务：`rosbridge_server`（ROS WebSocket）、`web_video_server`（HTTP 视频流）

通信协议总览

编号	接口名称	协议	方向
1	ROS WebSocket 连接	WebSocket	双向
2a	相机画面 - MJPEG 持续推流	HTTP 长连接（multipart）	服务端推流
2b	相机画面 - Snapshot 轮询	HTTP 短轮询（单帧）	客户端定时拉取
2c	仿真场景画面（固定话题）	HTTP 长连接（multipart）	服务端推流
3		ROS Topic（rosbridge）	前端 → 服务端

	发布任务目标（开始任务）		
4	取消任务	ROS Topic（rosbridge）	前端 → 服务端
5	订阅任务执行反馈	ROS Topic（rosbridge）	服务端 → 前端
6	订阅任务执行结果	ROS Topic（rosbridge）	服务端 → 前端

---

1. ROS WebSocket 连接

接口说明

页面挂载时通过 `roslibjs` 与 rosbridge_server 建立 WebSocket 长连接，所有 ROS Topic 的发布与订阅均通过此连接完成。断线后每 3 秒自动重连。

连接地址

代码块

```
ws://{host}:{rosPort}
```

参数说明

参数	类型	默认值	说明
host	string	`localhost`	rosbridge 服务器地址
rosPort	number	`9090`	rosbridge 监听端口

连接示例

```
代码块
1   ws://localhost:9090
```

连接状态事件

事件	触发时机	前端行为
`connection`	连接成功	`rosConnected = true`，初始化 AssemblyActionClient
`error`	连接出错	`rosConnected = false`
`close`	连接断开	`rosConnected = false`，3 秒后自动重连；若任务进行中则显示错误提示

---

2. 视频流接口（web_video_server）

接口说明

页面所有视频画面均通过 `web_video_server` 的 `/stream` 端点获取，核心区别在于**连接方式**和**话题**：

使用场景	连接方式	核心差异
相机画面 - MJPEG 模式	**长连接持续推流**	`<img>` 保持一条 HTTP 连接不断开，服务端持续推帧；低延迟，带宽消耗高
相机画面 - Snapshot 模式	**短轮询逐帧拉取**	前端定时器每隔 N 毫秒更换 `_t` 时间戳重新发起请求，取单帧；带宽可控，延迟略高
仿真场景画面	**长连接持续推流**	同 MJPEG 模式，话题固定为 `/vrep/camera/image`

请求地址

代码块

```
GET http://{host}:{serverPort}/stream
```

Base URL 参数说明

参数	类型	必填	说明
host	string	是	服务器地址，页面顶部可配置，默认 `localhost`
serverPort	number	是	web_video_server 端口，默认 `8080`

2a. 相机画面 - MJPEG 持续推流模式

单条 HTTP 连接保持不断开，服务端持续推送 multipart JPEG 帧，浏览器通过 `<img src>` 自动渲染。组件：`CameraView.vue`。

入参说明

参数	类型	必填	说明
topic	string	是	ROS 图像话题名称
type	string	是	流格式：`mjpeg` | `ros_compressed` | `vp8`
quality	number	否	MJPEG 压缩质量（0-100），默认 `80`
bitrate	number	否	VP8 码率（bps），仅 `type=vp8` 时有效，默认 `500000`

入参示例

代码块

```
# MJPEG（默认，质量 80）
GET http://localhost:8080/stream?topic=/camera/color/image_raw_left&type=mjpeg&quality=80

# ROS Compressed（订阅 /compressed 子话题）
GET http://localhost:8080/stream?topic=/camera/color/image_raw_left/compressed&type=ros_compressed

# VP8 编码流
GET http://localhost:8080/stream?topic=/camera/color/image_raw_left&type=vp8&bitrate=500000
```

出参说明

字段	类型	说明
-	`multipart/x-mixed-replace`	持续推送的 JPEG 帧序列，浏览器 `<img>` 元素直接解码渲染

连接状态（前端）

状态	触发条件	界面表现
loading	等待首帧，或图像 `naturalWidth = 0`	黄色圆点 + "连接中..." 遮罩，每 500ms 检测一次
connected	`naturalWidth > 0`（有实际像素）	绿色圆点，遮罩消失
error	`onerror` 事件触发，或 30s 内未收到首帧	红色圆点 + 错误遮罩 + 重试按钮

2b. 相机画面 - Snapshot 轮询模式

前端定时器每隔固定间隔重新请求一帧图片，通过 `_t` 时间戳参数防止浏览器缓存，实现受控帧率刷新。组件：`CameraViewSnapshot.vue`。

入参说明

参数	类型	必填	说明
topic	string	是	ROS 图像话题名称

_t	number	是	当前 Unix 时间戳（毫秒），每次请求不同值以绕过浏览器缓存

入参示例

代码块

```
GET http://localhost:8080/stream?topic=/camera/color/image_raw_left&_t=1710000000000
```

出参说明

字段	类型	说明
-	`image/jpeg`	单帧 JPEG 图像，每次请求后连接关闭

可选帧率

前端选项	轮询间隔	说明
10 FPS	100 ms	高频，适合实时观察

| 5 FPS | 200 ms | **默认**，平衡帧率与带宽 |

| 2 FPS | 500 ms | 低频，节省带宽 |

| 1 FPS | 1000 ms | 最低频，适合远程弱网 |

2c. 仿真场景画面（固定话题）

与 2a 完全相同的连接方式，话题固定为 V-REP 仿真相机，显示在"操作状态可视化"区域。

入参示例

代码块

```
GET http://localhost:8080/stream?topic=/vrep/camera/image&type=mjpeg
```

出参说明

字段	类型	说明
-	`multipart/x-mixed-replace`	MJPEG 持续推流；话题无数据时前端显示"等待仿真画面..."占位符

---

5. 发布任务目标（开始任务）

接口说明

用户点击"开始任务"时，通过 rosbridge 向 ROS Topic 发布任务指令，由服务端的桥接节点接收后真正调用 Action Server。

Topic 信息

字段	值
Topic 名称	`/assemble_goal`
消息类型	`std_msgs/msg/String`
方向	前端发布 → 服务端订阅

入参说明（消息 data 字段，JSON 字符串）

字段	类型	必填	说明
goal_id	string	否	前端生成的唯一 ID（格式：`goal_{timestamp}`），供日志追踪，服务端可忽略
command	string	是	任务指令，固定值 `"start"`

入参示例

代码块

```
{
  "data": "{\"goal_id\":\"goal_1710000000000\",\"command\":\"start\"}"
}
```

> 实际通过 `ROSLIB.Message({ data: JSON.stringify({...}) })` 构造，`data` 字段为 JSON 字符串。

6. 取消任务

接口说明

用户点击"结束任务"时，向取消 Topic 发布空消息，服务端收到后中止当前 Action。

Topic 信息

字段	值
Topic 名称	`/assemble_act/cancel`
消息类型	`std_msgs/msg/Empty`
方向	前端发布 → 服务端订阅

入参说明

无入参（Empty 消息）。

入参示例

代码块

```
{}
```

---

7. 订阅任务执行反馈（Feedback）

接口说明

任务执行过程中，服务端实时推送当前执行步骤和节点状态，前端据此更新任务流程图的高亮步骤。

Topic 信息

字段	值
Topic 名称	`assemble_feedback`
消息类型	`assemble_msg/msg/AssembleFeedback`
方向	服务端发布 → 前端订阅

出参说明

字段	类型	说明
current_node	int	当前正在执行的任务步骤索引（0-based），对应任务步骤列表中的位置
node_status	int	当前节点状态：`0` = 运行中，`1` = 成功，`-1` = 失败

出参示例

代码块

```
{
```

```
  "current_node": 2,
  "node_status": 0
}
```

任务步骤列表（current_node 对应关系）

current_node	任务描述
0	将桌子上的工件放到固定台上
1	拿取工具
2	工件装配
3	取下工具
4	合上盖子
5	按下固定台两侧按钮
6	打开盖子
7	取下固定台上的工件放到桌子上

前端消费逻辑

代码块

```
当 0 ≤ current_node < taskSteps.length 时:
  - currentStep = current_node     → 高亮对应步骤
  - nodeStatus  = node_status      → 更新步骤状态徽章
```

> **兼容性说明**：前端同时兼容 `std_msgs/msg/String`（data 字段为 JSON 字符串）和直接消息类型，rosbridge 返回哪种格式均可正常解析。

8. 订阅任务执行结果（Result）

接口说明

任务完成（成功或失败）后，服务端推送最终结果，前端据此更新任务状态并清理订阅。

Topic 信息

字段	值
Topic 名称	`assemble_result`
消息类型	`assemble_msg/msg/AssembleResult`
方向	服务端发布 → 前端订阅

出参说明

字段	类型	说明
final_result	bool	任务最终执行结果：`true` = 成功，`false` = 失败
fail_reason	string	失败原因描述，任务成功时为空字符串 `""`

出参示例

成功

代码块

```
{
  "final_result": true,
  "fail_reason": ""
}
```

失败

代码块

```
{
  "final_result": false,
  "fail_reason": "抓取工件失败，夹爪未检测到物体"
}
```

前端消费逻辑

代码块

```
final_result = true:
  - taskRunning = false
  - nodeStatus = 1（成功）
  - currentStep = taskSteps.length（所有步骤标记完成）

final_result = false:
  - taskRunning = false
  - nodeStatus = -1（失败）
  - failReason = fail_reason（显示错误提示条）
  - 弹出错误 Toast: `任务执行失败: {fail_reason}`
```

> 遥操作页面复用视频流能力（同 [第 2 节](#2-视频流接口web_video_server)），另有以下专属接口：

通信协议总览

编号	接口名称	协议	方向
T1	视频流（相机 / 仿真）	HTTP（web_video_server）	服务端推流 / 客户端拉取
T2	启动遥操作	ROS Topic（rosbridge）	前端 → 服务端
T3	停止遥操作	ROS Topic（rosbridge）	前端 → 服务端
T4	订阅遥操采集状态	ROS Topic（rosbridge）	服务端 → 前端
T5	订阅实时视频列表	ROS Topic（rosbridge）	服务端 → 前端

T1 视频流：

视频流接口（web_video_server）

接口说明

页面所有视频画面均通过 `web_video_server` 的 `/stream` 端点获取，核心区别在于**连接方式**和**话题**：

使用场景	连接方式	核心差异
相机画面 - MJPEG 模式	**长连接持续推流**	`<img>` 保持一条 HTTP 连接不断开，服务端持续推帧；低延迟，带宽消耗高
相机画面 - Snapshot 模式	**短轮询逐帧拉取**	前端定时器每隔 N 毫秒更换 `_t` 时间戳重新发起请求，取单帧；带宽可控，延迟略高
仿真场景画面	**长连接持续推流**	同 MJPEG 模式，话题固定为 `/vrep/camera/image`

请求地址

代码块

```
GET http://{host}:{serverPort}/stream
```

Base URL 参数说明

参数	类型	必填	说明
host	string	是	服务器地址，页面顶部可配置，默认 `localhost`
serverPort	number	是	web_video_server 端口，默认 9999

2a. 相机画面 - MJPEG 持续推流模式

单条 HTTP 连接保持不断开，服务端持续推送 multipart JPEG 帧，浏览器通过 `<img src>` 自动渲染。组件：`CameraView.vue`。

入参说明

参数	类型	必填	说明
topic	string	是	ROS 图像话题名称
type	string	是	流格式：`mjpeg` | `ros_compressed` | `vp8`
quality	number	否	MJPEG 压缩质量（0-100），默认 `80`
bitrate	number	否	VP8 码率（bps），仅 `type=vp8` 时有效，默认 `500000`

入参示例

代码块

```
# MJPEG（默认，质量 80）
GET http://localhost:8080/stream?topic=/camera/color/image_raw_left&type=mjpeg&quality=80

# ROS Compressed（订阅 /compressed 子话题）
GET http://localhost:8080/stream?topic=/camera/color/image_raw_left/compressed&type=ros_compressed

# VP8 编码流
GET http://localhost:8080/stream?topic=/camera/color/image_raw_left&type=vp8&bitrate=500000
```

出参说明

字段	类型	说明
-	`multipart/x-mixed-replace`	持续推送的 JPEG 帧序列，浏览器 `<img>` 元素直接解码渲染

连接状态（前端）

状态	触发条件	界面表现
loading	等待首帧，或图像 `naturalWidth = 0`	黄色圆点 + "连接中..." 遮罩，每 500ms 检测一次
connected	`naturalWidth > 0` （有实际像素）	绿色圆点，遮罩消失
error	`onerror` 事件触发，或 30s 内未收到首帧	红色圆点 + 错误遮罩 + 重试按钮

T2. 启动遥操作

接口说明

前端点击"启动遥操作"后，通过 rosbridge 向 ROS Topic 发布启动指令，服务端接收后执行 Shell 脚本，依次启动 VR 手柄监听、数据录制保存等相关线程。

Topic 信息

字段	值
Topic 名称	`/teleop_goal`
消息类型	`std_msgs/msg/String`
方向	前端发布 → 服务端订阅

入参说明（消息 data 字段，JSON 字符串）

字段	类型	必填	说明
goal_id	string	否	前端生成的唯一 ID（格式：`goal_{timestamp}`），供日志追踪，服务端可忽略
command	string	是	任务指令，固定值 `"start"`

入参示例

代码块

```
{
  "data": "{\"goal_id\":\"goal_1710000000000\",\"command\":\"start\"}"
}
```

> 实际通过 `ROSLIB.Message({ data: JSON.stringify({...}) })` 构造，`data` 字段为 JSON 字符串。

T3. 停止遥操作

接口说明

前端点击"停止遥操作"后，向停止 Topic 发布空消息，服务端收到后终止所有遥操相关线程（VR 监听、数据保存等），释放资源并将状态重置为空闲。

Topic 信息

字段	值
Topic 名称	`/teleop_act/cancel`
消息类型	`std_msgs/msg/Empty`
方向	前端发布 → 服务端订阅

入参说明

无入参（Empty 消息）。

入参示例

代码块

```
{}
```

T4. 订阅遥操采集状态反馈

接口说明

前端订阅此 Topic，服务端在遥操运行期间实时推送当前采集状态，前端据此更新页面状态指示器（空闲 / 采集中 / 异常）。

Topic 信息

字段	值
Topic 名称	`teleop_feedback`
消息类型	`teleop_msg/msg/TeleopFeedback`
方向	服务端发布 → 前端订阅

出参说明

字段	类型	说明
status	int	当前采集状态：`0` = 空闲，`1` = 采集中，`-1` = 异常
duration	number	当次采集已持续时长（秒），空闲时为 `0`
frame_count	number	当次已采集帧数，空闲时为 `0`

status 枚举

值	含义
`0`	空闲，未运行任何遥操线程
`1`	采集中，VR 及录制线程均正常运行
`-1`	异常，线程启动失败或中途崩溃

出参示例

采集中

代码块

```
{
  "status": 1,
  "duration": 42,
  "frame_count": 420
}
```

空闲

代码块

```
{
  "status": 0,
  "duration": 0,
  "frame_count": 0
}
```

前端消费逻辑

```
status = 0  → 状态指示器显示"空闲"，采集按钮可点击
status = 1  → 状态指示器显示"采集中"，展示 duration 和 frame_count
status = -1 → 状态指示器显示"异常"，弹出错误提示
```

> **兼容性说明**：前端同时兼容 `std_msgs/msg/String`（data 字段为 JSON 字符串）和直接消息类型，rosbridge 返回哪种格式均可正常解析。

T5. 订阅实时视频列表

接口说明

前端订阅此 Topic，服务端推送当前可用的实时视频流列表（相机名称 + 播放链接）。前端收到后动态渲染视频播放器列表，每路视频的播放链接格式与 T1 视频流接口完全一致，可直接赋给 `<img src>` 进行播放。

服务端可在相机设备上线/下线时重新发布此 Topic，前端订阅后自动刷新列表。

Topic 信息

字段	值
Topic 名称	`teleop_camera_list`
消息类型	`teleop_msg/msg/CameraList`
方向	服务端发布 → 前端订阅

出参说明

消息为数组结构，每个元素描述一路视频流：

字段	类型	说明
cameras	array	视频流列表
cameras[].id	number	相机唯一 ID
cameras[].name	string	相机展示名称，如 `"相机 1 - 前视"`
cameras[].topic	string	对应 ROS 图像话题，如 `/camera/color/image_raw_left`
cameras[].stream_url	string	完整 MJPEG 推流链接，可直接赋给 `<img src>`（参见 T1-2a）
cameras[].snapshot_url	string	快照基础链接，前端使用时自行追加 `&_t={timestamp}`（参见 T1-2b）
cameras[].status	string	当前状态：`"online"` = 正常推流，`"offline"` = 设备离线

出参示例

代码块

```
{
  "cameras": [
    {
      "id": 1,
      "name": "相机 1 - 前视",
      "topic": "/camera/color/image_raw_left",
      "stream_url": "http://localhost:8080/stream?topic=/camera/color/image_raw_left&type=mjpeg&quality=80",
      "snapshot_url": "http://localhost:8080/stream?topic=/camera/color/image_raw_left",
```

```
        "status": "online"
      },
      {
        "id": 2,
        "name": "相机 2 - 后视",
        "topic": "/camera/color/image_raw_right",
        "stream_url": "http://localhost:8080/stream?topic=/camera/color/image_raw_right&type=mjpeg&quality=80",
        "snapshot_url": "http://localhost:8080/stream?topic=/camera/color/image_raw_right",
        "status": "online"
      },
      {
        "id": 3,
        "name": "相机 3 - 左视",
        "topic": "/camera3/image_raw",
        "stream_url": "http://localhost:8080/stream?topic=/camera3/image_raw&type=mjpeg&quality=80",
        "snapshot_url": "http://localhost:8080/stream?topic=/camera3/image_raw",
        "status": "offline"
      }
    ]
}
```

前端消费逻辑

代码块

```
收到消息后:
  - 过滤 status = "online" 的条目渲染视频播放器
  - MJPEG 模式: 直接将 stream_url 赋给 <img src>
  - Snapshot 模式: 将 snapshot_url + "&_t={Date.now()}" 定时赋给 <img src>
  - status = "offline" 的条目显示离线占位符
```

遥操作页面接口文档

> 遥操作页面复用视频流能力（同 [第 2 节](#2-视频流接口web_video_server)），另有以下专属接口。T2/T3 为 HTTP 请求，T4 为 WebSocket 推送，T5 为 HTTP 请求；**不依赖 rosbridge**，需后端提供对应 HTTP API 与 WebSocket 端点。

通信协议总览

编号	接口名称	协议	方向
T1	视频流（相机 / 仿真）	HTTP（web_video_server）	服务端推流 / 客户端拉取
T2	启动遥操作	HTTP POST	前端 → 服务端
T3	停止遥操作	HTTP POST	前端 → 服务端
T4	订阅遥操采集状态	WebSocket	服务端 → 前端
T5	获取实时视频列表	HTTP GET	前端 → 服务端

> **T1 视频流**：与实时监控页面完全相同，参见上方第 2 节，此处不再重复说明。

T2. 启动遥操作

接口说明

前端点击"启动遥操作"后，通过 HTTP POST 请求发送启动指令，服务端接收后执行 Shell 脚本，依次启动 VR 手柄监听、数据录制保存等相关线程。

请求信息

项目	值
请求方式	POST
请求地址	`http://{host}:{apiPort}/api/teleop/start`
Content-Type	`application/json`

Base URL 参数说明

参数	类型	必填	说明
host	string	是	服务器地址，与页面配置一致
apiPort	number	是	遥操作 API 服务端口（可与 web_video_server 同机不同端口，或由网关统一）

请求体（Body）入参

字段	类型	必填	说明
goal_id	string	否	前端生成的唯一 ID（格式：`goal_{timestamp}`），供日志追踪，服务端可忽略

command	string	是	任务指令，固定值 `"start"`

请求示例

代码块

```
POST http://localhost:8080/api/teleop/start
Content-Type: application/json

{
  "goal_id": "goal_1710000000000",
  "command": "start"
}
```

响应说明

字段	类型	说明
code	number	状态码：`0` 成功，非 0 失败
message	string	提示信息
data	object	可选，扩展数据

响应示例

成功（HTTP 200）

代码块

```
{
  "code": 0,
```

```
  "message": "ok",
  "data": null
}
```

失败（HTTP 200 或 4xx/5xx）

代码块

```
{
  "code": 1,
  "message": "遥操作服务已启动或启动失败原因描述",
  "data": null
}
```

T3. 停止遥操作

接口说明

前端点击"停止遥操作"后，通过 HTTP POST 请求通知服务端，服务端收到后终止所有遥操相关线程（VR 监听、数据保存等），释放资源并将状态重置为空闲。

请求信息

项目	值
请求方式	POST
请求地址	`http://{host}:{apiPort}/api/teleop/stop`

Content-Type	`application/json`（Body 可为空）

Base URL 参数说明：同 T2。

请求体（Body）

无必填字段，可传空对象 `{}` 或不传 Body。

请求示例

代码块

```
POST http://localhost:8080/api/teleop/stop
Content-Type: application/json

{}
```

响应说明

字段	类型	说明
code	number	状态码：`0` 成功，非 0 失败
message	string	提示信息
data	object	可选，扩展数据

响应示例

成功（HTTP 200）

代码块

```
{
  "code": 0,
  "message": "ok",
  "data": null
}
```

T4. 订阅遥操采集状态（WebSocket）

接口说明

前端建立 WebSocket 连接后，服务端在遥操运行期间通过该连接实时推送当前采集状态，前端据此更新页面状态指示器（空闲 / 采集中 / 异常）。

连接信息

项目	值
协议	WebSocket
连接地址	`ws://{host}:{apiPort}/api/teleop/feedback` 或 `wss://...`（若启用 TLS）

参数说明：`host`、`apiPort` 与 T2/T3 一致；建立连接时无需 query 参数，连接成功后服务端会按需推送消息。

入参（客户端 → 服务端）

建立连接后，客户端一般无需发送业务报文；若服务端约定支持「订阅/取消」等控制帧，则按服务端约定格式发送（此处默认连接即订阅，无额外入参）。

出参（服务端 → 前端，单条消息 JSON）

服务端推送的每条消息为 JSON 对象，字段如下：

字段	类型	说明
status	int	当前采集状态：`0` = 空闲，`1` = 采集中，`-1` = 异常
duration	number	当次采集已持续时长（秒），空闲时为 `0`
frame_count	number	当次已采集帧数，空闲时为 `0`

status 枚举

值	含义
`0`	空闲，未运行任何遥操线程
`1`	采集中，VR 及录制线程均正常运行
`-1`	异常，线程启动失败或中途崩溃

出参示例

采集中

代码块

```
{
  "status": 1,
  "duration": 42,
  "frame_count": 420
}
```

空闲

代码块

```
{
  "status": 0,
  "duration": 0,
  "frame_count": 0
}
```

前端消费逻辑

代码块

```
连接建立后监听 onmessage，解析 JSON：
  status = 0  → 状态指示器显示"空闲"，采集按钮可点击
  status = 1  → 状态指示器显示"采集中"，展示 duration 和 frame_count
  status = -1 → 状态指示器显示"异常"，弹出错误提示
```

T5. 获取实时视频列表

接口说明

前端通过 HTTP GET 请求获取当前可用的实时视频流列表（相机名称 + 播放链接）。响应中的播放链接格式与 T1 视频流接口完全一致，可直接赋给 `<img src>` 进行播放。

前端可在进入页面、定时轮询或用户点击刷新时调用本接口，以在相机设备上线/下线时更新列表。

请求信息

项目	值
请求方式	GET
请求地址	`http://{host}:{apiPort}/api/teleop/cameras`

Base URL 参数说明：同 T2。

入参（Query）

无必填参数；若有可选参数（如 `format=snapshot` 等），由服务端在接口说明中补充。

响应说明（HTTP 200 Body）

字段	类型	说明
code	number	状态码：`0` 成功，非 0 失败

message	string	提示信息
data	object	见下表
data.cameras	array	视频流列表，每项结构见下

data.cameras[ ] 单元素

字段	类型	说明
id	number	相机唯一 ID
name	string	相机展示名称，如 `"相机 1 - 前视"`
topic	string	对应 ROS 图像话题，如 `/camera/color/image_raw_left`
stream_url	string	完整 MJPEG 推流链接，可直接赋给 `<img src>`（参见 T1-2a）
snapshot_url	string	快照基础链接，前端使用时自行追加 `&_t={timestamp}`（参见 T1-2b）
status	string	当前状态：`"online"` = 正常推流，`"offline"` = 设备离线

响应示例

代码块

```
{
  "code": 0,
  "message": "ok",
  "data": {
```

```
    "cameras": [
      {
        "id": 1,
        "name": "相机 1 - 前视",
        "topic": "/camera/color/image_raw_left",
        "stream_url": "http://localhost:8080/stream?topic=/camera/color/image_raw_left&type=mjpeg&quality=80",
        "snapshot_url": "http://localhost:8080/stream?topic=/camera/color/image_raw_left",
        "status": "online"
      },
      {
        "id": 2,
        "name": "相机 2 - 后视",
        "topic": "/camera/color/image_raw_right",
        "stream_url": "http://localhost:8080/stream?topic=/camera/color/image_raw_right&type=mjpeg&quality=80",
        "snapshot_url": "http://localhost:8080/stream?topic=/camera/color/image_raw_right",
        "status": "online"
      },
      {
        "id": 3,
        "name": "相机 3 - 左视",
        "topic": "/camera3/image_raw",
        "stream_url": "http://localhost:8080/stream?topic=/camera3/image_raw&type=mjpeg&quality=80",
        "snapshot_url": "http://localhost:8080/stream?topic=/camera3/image_raw",
        "status": "offline"
      }
    ]
  }
}
```

前端消费逻辑

代码块

```
请求 GET /api/teleop/cameras 后:
  - 解析 data.cameras, 过滤 status = "online" 的条目渲染视频播放器
  - MJPEG 模式: 直接将 stream_url 赋给 <img src>
  - Snapshot 模式: 将 snapshot_url + "&_t=" + Date.now() 定时赋给 <img src>
```

```
  - status = "offline" 的条目显示离线占位符
  - 可定时轮询本接口以刷新列表
```